

ЛАБОРАТОРНА РОБОТА 4

Тема: РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Етап 1 Підготовка (самостійно):

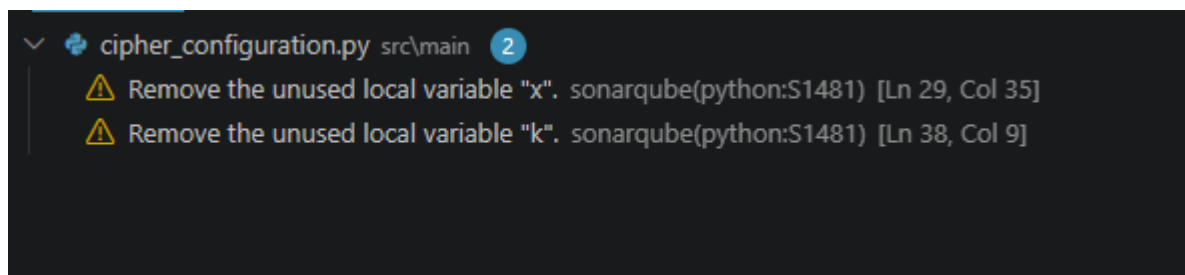
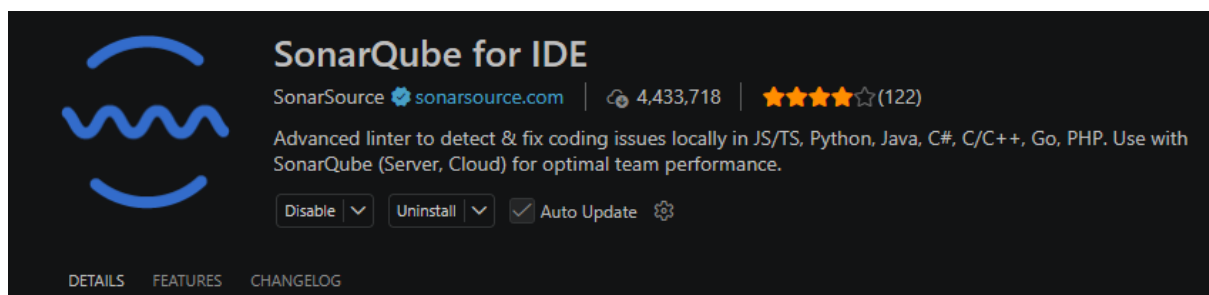
1) Код модуля з ЛР3:

https://github.com/Ruslan62/bse-lr1-Usatenko/blob/main/src/main/cipher_configuration.py

Тести з ЛР3:

https://github.com/Ruslan62/bse-lr1-Usatenko/blob/main/src/test/test_cipher.py

2) Встановити SonarQube for IDE (SonarLint) у VS Code. Запустити аналіз свого коду та зафіксувати початкову кількість issues (скріншот «до»):



3) Запустити лінтер (Ruff для Python: `ruff check src/`). Зафіксувати кількість попереджень:

```
39 | ...      # Розрахунок кі
40 | ...      blocks_count =
    |
help: Remove assignment to

Found 5 errors.
[*] 2 fixable with the `--
```

Етап 2. Code Review (взаємний).

4) Репозиторій одnogрупника:

<https://github.com/OlehSheremet/bse-lr1-Sheremet>

Код модуля одnogрупника з ЛР3:

<https://github.com/OlehSheremet/bse-lr1-Sheremet/blob/review-ruslan-usatenko/src/main/main.py>

5) Посилання на гілку:

<https://github.com/OlehSheremet/bse-lr1-Sheremet/tree/review-ruslan-usatenko/src>

6) Провести Code Review: ретельно проаналізувати код одnogрупника та виявити щонайменше 5 проблем якості коду.

№	Рядок коду	Проблема	Категорія	Рекомендація
1	78	Число 3000 використовується як жорстко прописаний ліміт бітрейту для безкоштовних акаунтів без будь-якого контексту.	Магічні числа	Слід винести його наверх файлу або в константи класу з чітким ім'ям, наприклад: <code>MAX_FREE_BITRATE_KBPS = 3000</code> .
2	122	Код створення об'єкта <code>HistoryRecord</code> , додавання його до списку <code>self.history</code> та	Дублювання коду	Застосувати операцію <code>Consolidate Duplicate</code> .

		виклик <code>save_locally()</code> дублюється майже ідентично в обох гілках.		Винести повторювану логіку запису історії в окремий допоміжний метод класу <code>User</code> , наприклад: <code>_add_history_record(self, file, status)</code> .
3	20	Назва аргументу <code>format</code> збігається з назвою вбудованої функції <code>Python format()</code> , через що оригінальна функція перекривається всередині цього методу.	Погане іменування	Треба перейменувати параметр та властивість класу на <code>file_format</code>
4	15	Число 30 означає тривалість підписки у днях, але записане як жорсткий літерал.	Магічні числа	Створити константу класу або глобальну константу <code>SUBSCRIPTION_DURATION_DAYS = 30</code>
5	30	Розмір буфера завантаження (5 МБ) зашитий прямо всередині методу. Якщо знадобиться змінити розмір шматка для оптимізації мережі або написання тестів, зробити це буде важко.	Магічні числа	Винеси це значення як константу класу за замовчуванням (наприклад, <code>DEFAULT_CHUNK_SIZE</code>).

7) Кожну проблему задокументувати як inline-коментар у PR:

<https://github.com/OlehSheremet/bse-lr1-Sheremet/pull/3#pullrequestreview-4436292817>

Етап 3. Рефакторинг (самостійно):

8) Отримати результати Code Review від однокласника.

<https://github.com/Ruslan62/bse-lr1-Usatenko/pull/2>

9) Виконати щонайменше 3 рефакторинг-операції за результатами отриманого review. Для кожної операції задокументувати: «БУЛО → СТАЛО → ЧОМУ» (обґрунтування).

1. Було:

```
import os
import sys
class CipherConfiguration:
```

Стало:

```
class CipherConfiguration:
```

Чому: Модулі os та sys були імпортовані на початку файлу, проте жодна їхня функція чи метод не використовуються в логіці класу CipherConfiguration.

2. Було:

```
def generate_key(self, seed_phrase: str) -> str:
    if not seed_phrase or len(seed_phrase.strip()) < 3:
        raise ValueError("Фраза-зерно (seed) має містити хоча б 3 символи.")
```

Стало:

```
def generate_key(self, seed_phrase: str) -> str:
    if not seed_phrase or len(seed_phrase.strip()) < MIN_SEED_LENGTH:
        raise ValueError(f"Фраза-зерно (seed) має містити хоча б
{MIN_SEED_LENGTH} символи.")
```

Чому: Число 3 використовувалося в коді як жорстко зашитий літерал. Це ускладнює супровід програми. Винесення цього значення в глобальну константу MIN_SEED_LENGTH робить код самодокументованим і дозволяє змінити правила безпеки (наприклад, збільшити мінімальну довжину до 8 символів).

3. Було:

```
self.key_size = key_size; x=1
# У методі generate_key:
k = seed_phrase
```

Стало:

```
self.key_size = key_size
```

(рядок з k повністю видалено)

Чому: Локальні змінні $x=1$ та k створювалися в коді, але жодним чином не використовувалися в подальших обчисленнях. Це є ознакою запаху коду Dead Code.

10) Після кожного кроку рефакторингу запускати тести з ЛР 03 та переконуватися, що вони залишаються green (регресійне тестування).
Після першого кроку:

```
===== tests coverage =====
----- coverage: platform win32, python 3.13.7-final-0 -----
Name                               Stmts  Miss  Cover
-----
src\main\cipher_configuration.py    50      4    92%
TOTAL                               50      4    92%
Required test coverage of 80% reached. Total coverage: 92.00%
```

Після другого кроку:

```
===== tests coverage =====
----- coverage: platform win32, python 3.13.7-final-0 -----
Name                               Stmts  Miss  Cover
-----
src\main\cipher_configuration.py    50      4    92%
TOTAL                               50      4    92%
Required test coverage of 80% reached. Total coverage: 92.00%
===== 12 passed in 0.11s =====
PS C:\Users\ASUS\bse-lr1-Ustatenko>
```

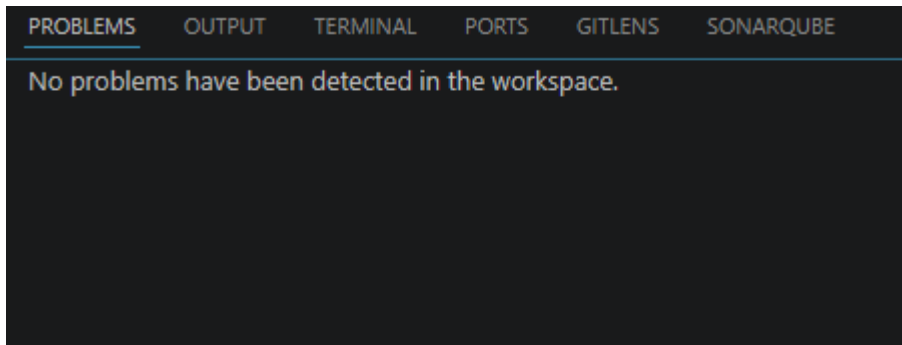
Після третього кроку:

```
===== tests coverage =====
----- coverage: platform win32, python 3.13.7-final-0 -----
Name                               Stmts  Miss  Cover
-----
src\main\cipher_configuration.py    50      4    92%
TOTAL                               50      4    92%
Required test coverage of 80% reached. Total coverage: 92.00%
===== 12 passed in 0.11s =====
PS C:\Users\ASUS\bse-lr1-Ustatenko>
```

11) лінтер Ruff

```
PS C:\Users\ASUS\bse-lr1-Ustate
All checks passed!
PS C:\Users\ASUS\bse-lr1-Ustate
```

SonarLint:



12) Цикломатична складність (CC):

```
src/main/cipher_configuration.py
  C 3:0 CipherConfiguration - B (6)
  M 54:4 CipherConfiguration.process_batch_data - B (6)
  M 6:4 CipherConfiguration.__init__ - A (5)
  M 31:4 CipherConfiguration.generate_key - A (5)
PS C:\Users\ASUS\bse-lr1-Usatenko>
```

Індекс підтримки коду (MI):

```
PS C:\Users\ASUS\bse-lr1-Usatenko> python -m radon mi src/main/cipher_configuration.py -s
src/main/cipher_configuration.py - A (70.45)
PS C:\Users\ASUS\bse-lr1-Usatenko>
```

Порівняння метрик до і після рефакторингу:

Метрика	До	Після
Цикломатична складність (max)	6	6
SonarLint issues	2	0
Ruff warnings	5	0
Тести (pass / total)	12/12	12/12

Бонусне завдання: використати GitHub Copilot Chat для отримання пояснення підозрілого фрагмента коду (Explain this code) або пропозиції рефакторингу (Suggest refactoring):

```

def generate_key(self, seed_phrase: str) -> str:
    """
    Генерація шаблону ключа - створення детермінованої текстової послідовності на основі користувацької секретної фрази (seed).
    """
    if not seed_phrase or len(seed_phrase.strip()) < MIN_SEED_LENGTH:
        raise ValueError(f"Фраза-зерно (seed) має містити хоча б {MIN_SEED_LENGTH} символи.")
    # Розрахунок кількості структурних блоків ключа залежно від його загальної бітової довжини
    blocks_count = self.key_size // 64
    if blocks_count == 0:
        blocks_count = 1 # Запобігання нульовій довжині для низькобітного стандарту DES

    generated_key = ""
    cleaned_seed = seed_phrase.strip().upper()

    # Покрокове формування унікальних сегментів ключа за допомогою математичного зміщення фрази
    for i in range(1, blocks_count + 1):
        block_part = f"{cleaned_seed[:MIN_SEED_LENGTH]}{i * 7}"
        generated_key += block_part + "-"

    final_key = generated_key.rstrip("-")
    cleaned_seed = self._validate_and_clean_seed(seed_phrase)
    blocks_count = self._calculate_blocks_count()
    final_key = self._build_key_from_blocks(cleaned_seed, blocks_count)

    self.history_logs.append(f"Згенеровано шаблон ключа: {final_key}")
    return final_key

def _validate_and_clean_seed(self, seed_phrase: str) -> str:
    """Валідація та очищення seed-фрази."""
    if not seed_phrase or len(seed_phrase.strip()) < MIN_SEED_LENGTH:
        raise ValueError(f"Фраза-зерно (seed) має містити хоча б {MIN_SEED_LENGTH} символи.")
    return seed_phrase.strip().upper()

def _calculate_blocks_count(self) -> int:
    """Розрахунок кількості структурних блоків ключа."""
    blocks_count = self.key_size // 64
    return blocks_count if blocks_count > 0 else 1

def _build_key_from_blocks(self, cleaned_seed: str, blocks_count: int) -> str:
    """Формування унікальних сегментів ключа."""
    key_blocks = [
        f"{cleaned_seed[:MIN_SEED_LENGTH]}{i * 7}"
        for i in range(1, blocks_count + 1)
    ]
    return "-".join(key_blocks)

```

Підсумкова рефлексія:

Під час виконання лабораторної роботи я отримав практичний досвід роботи як у ролі автора коду, так і в ролі рецензента. Коли я самостійно шукав помилки у коді однокласника та залишав йому коментарі, це допомогло мені краще засвоїти які бувають типові проблеми якості коду. Наступне виправлення мого власного коду за отриманими зауваженнями показало, що програма має бути не просто робочою, а й легкою для читання та супроводу. Ця практика дала чітке розуміння того, як взаємодія між розробниками покращує якість кінцевого продукту.